

# Tracklistd

## Guia de Setup do Ambiente de Desenvolvimento

### Visão Geral

O Tracklistd é composto por dois repositórios separados:

- **api**: back-end em Spring Boot (Java), executado via wrapper Maven (`./mvnw`).
- **front**: front-end em React Native com Expo, executado via `npx expo start`.

Antes de rodar qualquer um dos dois, é necessário baixar as dependências do projeto e configurar as variáveis de ambiente (`.env`) correspondentes.

### 1 Pré-requisitos

- Node.js (LTS) e npm instalados, para o **front**.
- JDK compatível com o projeto instalado, para a **api** (o Maven em si não precisa estar instalado globalmente, pois o projeto usa o wrapper `mvnw`).
- Uma instância de banco de dados (MySQL/MariaDB) acessível localmente, geralmente via Docker, para a **api**.
- Conta de acesso ao Firebase Console e ao Spotify for Developers, para gerar as credenciais (ver Seção 4).

### 2 Repositório api (Spring Boot)

#### 2.1 Instalar dependências

O wrapper do Maven baixa as dependências declaradas no `pom.xml` automaticamente na primeira execução. Para forçar o download sem rodar a aplicação:

```
./mvnw dependency:resolve
```

#### 2.2 Rodar a aplicação

Com o `.env` configurado (Seção 4.2) e o banco de dados acessível:

```
./mvnw spring-boot:run
```

Esse comando sobe a api usando apenas o `application.properties` padrão. Para escolher um profile específico, veja a Seção 2.3.

## 2.3 Profiles do Spring Boot

A api possui múltiplos arquivos de configuração em `src/main/resources/`, um para cada profile:

- `application.properties` – configuração base, sempre carregada.
- `application-dev.properties` – profile de desenvolvimento local.
- `application-populate.properties` – profile usado para popular o banco com dados iniciais/seed.
- `application-prod.properties` – profile de produção.

Quando um profile é ativado, o Spring Boot carrega o `application.properties` normalmente e, em seguida, sobrepõe/complementa com as chaves do arquivo específico do profile (`application-{profile}.properties`).

Para ativar um profile ao rodar via Maven wrapper, use a flag `-Dspring-boot.run.profiles`:

```
# Profile de desenvolvimento
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev

# Profile de populacao do banco
./mvnw spring-boot:run -Dspring-boot.run.profiles=populate

# Profile de producao
./mvnw spring-boot:run -Dspring-boot.run.profiles=prod
```

Alternativamente, o profile também pode ser definido por variável de ambiente antes de subir a aplicação, o que é útil em pipelines de CI/CD ou containers:

```
export SPRING_PROFILES_ACTIVE=prod
./mvnw spring-boot:run
```

Rodar sem nenhuma flag de profile (`./mvnw spring-boot:run`) usa apenas o `application.properties`, sem nenhum dos overrides de `dev`, `populate` ou `prod`.

## 3 Repositório front (React Native / Expo)

### 3.1 Instalar dependências

```
npm install
```

### 3.2 Rodar a aplicação

Com o `.env` configurado (Seção 4.1):

```
npx expo start
```

## 4 Variáveis de ambiente (.env)

Cada repositório possui seu próprio arquivo `.env` na raiz, que **não deve ser versionado** (deve constar no `.gitignore`). Abaixo estão as chaves esperadas em cada um e como obter os respectivos valores. Os valores em si não são reproduzidos neste documento por segurança.

## 4.1 .env do front

**Chaves do Firebase** (EXPO\_PUBLIC\_FIREBASE\_API\_KEY, EXPO\_PUBLIC\_FIREBASE\_AUTH\_DOMAIN, EXPO\_PUBLIC\_FIREBASE\_PROJECT\_ID, EXPO\_PUBLIC\_FIREBASE\_STORAGE\_BUCKET, EXPO\_PUBLIC\_FIREBASE\_MESSAGING\_SENDER\_ID, EXPO\_PUBLIC\_FIREBASE\_APP\_ID, EXPO\_PUBLIC\_FIREBASE\_MEASUREMENT\_ID):

1. Acesse o Firebase Console e selecione o projeto **tracklistd**.
2. Vá em **Configurações do projeto** (ícone de engrenagem) → **Geral**.
3. Na seção **Seus apps**, selecione o app Web registrado (ou crie um, se ainda não existir).
4. O bloco de configuração do SDK (**firebaseConfig**) exibirá todos esses valores prontos para copiar.

**Client IDs do Google Sign-In** (EXPO\_PUBLIC\_WEB\_CLIENT\_ID, EXPO\_PUBLIC\_IOS\_CLIENT\_ID):

1. Acesse o Google Cloud Console, no mesmo projeto vinculado ao Firebase.
2. Vá em **APIs & Serviços** → **Credenciais**.
3. Em **IDs de cliente OAuth 2.0**, localize o cliente do tipo *Web application* (para WEB\_CLIENT\_ID) e o cliente do tipo *iOS* (para IOS\_CLIENT\_ID).
4. Caso não existam, crie-os pelo próprio Firebase Authentication (Método de login → Google), que gera esses clientes automaticamente.

## 4.2 .env da api

**Credenciais de banco de dados** (DB\_HOST, DB\_PORT, DB\_NAME, DB\_ROOT\_PASSWORD, DB\_USERNAME, DB\_PASSWORD):

Não são obtidas de nenhum serviço externo – são definidas por quem configura o ambiente local, normalmente através de um **docker-compose.yml** que sobe uma instância MySQL/-MariaDB. Os valores devem ser os mesmos usados na configuração do container do banco (host, porta exposta, nome do schema e credenciais do usuário).

**Credenciais do Spotify** (SPOTIFY\_CLIENT\_ID, SPOTIFY\_CLIENT\_SECRET):

1. Acesse o Spotify for Developers Dashboard.
2. Selecione o app registrado do Tracklistd (ou crie um novo app, caso necessário).
3. Em **Settings**, o *Client ID* fica visível diretamente; o *Client Secret* pode ser revelado clicando em **View client secret**.

**Credencial de administrador do Firebase** (FIREBASE\_JSON\_PATH):

1. No Firebase Console, vá em **Configurações do projeto** → **Contas de serviço**.
2. Clique em **Gerar nova chave privada**, o que baixa um arquivo JSON com as credenciais do Admin SDK.
3. Coloque esse arquivo em **src/main/resources/firebase/** dentro do repositório **api**.

4. Aponte `FIREBASE_JSON_PATH` no `.env` para o caminho relativo desse arquivo.

## 5 Observações de segurança

- Nunca faça commit dos arquivos `.env` nem do JSON do Firebase Admin SDK – ambos devem estar listados no `.gitignore`.
- O *Client Secret* do Spotify e as senhas de banco são credenciais sensíveis: evite compartilhá-las em chats, tickets ou repositórios públicos.
- Caso alguma credencial sensível seja exposta acidentalmente, revogue/regenere-a o quanto antes (no Spotify Dashboard, no Firebase Console ou trocando a senha do banco).